Syntax Anatomy: Objects and Classes

CMPT 211: Introduction to Software Development

Concordia University of Edmonton

Winter 2026

Anatomy of a Class Declaration

Let's dissect the exact syntax used to define our blueprint.

```
class ConcordiaStudent:
    """A blueprint representing a student at CUE."""
```

Line-by-Line Breakdown:

- `class`: The reserved Python keyword that tells the interpreter we are defining a new Object blueprint, not a standard function.
- `ConcordiaStudent`: The identifier (name) of our class. *Crucial Convention:* Class names must always use **PascalCase** (Capitalize every word, no underscores). This visually distinguishes them from standard variables.
- `:`: The colon opens a new indented code block. Everything indented below this belongs to the blueprint.
- `"""Docstring"""`: A triple-quoted string immediately following the declaration. It documents the class's purpose and is accessible via `help(ConcordiaStudent)`.

Anatomy of the Constructor (`__init__`)

The constructor is the setup engine. It runs the millisecond a new object is born.

```
    def __init__(self, name, student_id, year):
```

Field-by-Field Breakdown:

- `def`: We are defining a function, but because it is indented inside a `class`, it is officially called a **Method**.
- `__init__`: "Dunder" (Double-Underscore) init. The underscores tell Python: *"This is a special built-in method. Do not wait for the user to call it; run it automatically when the object is instantiated."*
- `self`: The mandatory first parameter. It is a blank name tag that Python will fill with the specific memory address of the object currently being built.
- `name, student_id, year`: The required parameters (inputs) the user must provide when creating a new student.

Anatomy of Attributes (Instance Variables)

Inside the constructor, we bind the incoming data to the object's permanent state.

```
        # 1. Dynamic Attributes (Passed by user)
        self.name = name
        self.student_id = student_id
        self.year = year

        # 2. Default Attributes (Hardcoded baseline)
        self.courses = []
        self.gpa = 0.0
```

The `self.` Prefix is Mandatory:

- `name` (without `self`) is just a temporary local variable. It will vanish from memory the moment `__init__` finishes running.
- `self.name` attaches the variable permanently to the object. It becomes an **Attribute** that can be accessed later (e.g., `print(alice.name)`).
- We can also create attributes with default values (like an empty `courses` list or a `0.0` GPA) that the user does not need to provide upfront.

Anatomy of a Custom Method (Behavior)

Once the object has data, we write methods to act upon that data. This is where **Abstraction** happens.

```
    def enroll_in_course(self, course_name):
        """Adds a course to the student's roster."""
        self.courses.append(course_name)
        print(f"{self.name} successfully enrolled in {course_name}.")
```

Field-by-Field Breakdown:

- `def enroll_in_course`: We define a new custom behavior. (Uses `snake_case`).
- `(self, course_name)`: We MUST pass `self` first, so the method knows *whose* course list to append to.
- `self.courses.append(...)`: We access the object's permanent state (`self.courses`) and mutate it.
- **The Abstraction:** The user just types `alice.enroll_in_course("CMPT 211")`. They do not need to know that a Python `list` is being used or how `.append()` works under the hood. The complexity is abstracted away into a simple command.

Anatomy of Encapsulation (Data Hiding)

We use specific syntax to enact **Encapsulation**—protecting the object from illegal modifications.

```
    def __init__(self, name, student_id):
        self.name = name
        # The single underscore prefix signals "PRIVATE"
        self._gpa = 0.0

    def update_gpa(self, new_gpa):
        # The Setter Method acts as a Firewall
        if type(new_gpa) is float and 0.0 <= new_gpa <= 4.0:
            self._gpa = new_gpa
        else:
            raise ValueError("GPA must be a float between 0.0 and 4.0")
```

The Syntax Mechanism: By prefixing `_gpa`, we warn other programmers not to write `alice._gpa = 99.9`. Instead, they are forced to route their data through the `update_gpa(new_gpa)` firewall, ensuring the system remains mathematically robust.

Anatomy of Inheritance

How do we tell Python that one class is a specialized version of another?

```
# 1. The syntax for declaring the Parent (Superclass)
class ITStudent(ConcordiaStudent):

    def __init__(self, name, student_id, year, server_access):
        # 2. The syntax for invoking the Parent's Constructor
        super().__init__(name, student_id, year)

        # 3. Setting Child-specific attributes
        self.major = "Information Technology"
        self.access_level = server_access
```

The Syntax Mechanism:

- `(ConcordiaStudent)`: Placing the parent's name in parentheses during the class declaration establishes the "IS-A" inheritance link.
- `super()`: A built-in function that fetches the Parent class. Calling `super().__init__()` tells the Parent to execute its setup code (setting up the name, ID, and default GPA) so we don't have to rewrite those lines.

Anatomy of Polymorphism (Method Overriding)

Polymorphism relies on **Method Overriding**—writing a method in a child class with the exact same name as a method in the parent class or sibling class.

```
class ITStudent(ConcordiaStudent):
    def access_lab(self): # Exact same method signature
        return f"Accessing Server Room with {self.access_level} privileges."

class CSStudent(ConcordiaStudent):
    def access_lab(self): # Exact same method signature
        return f"Logging into Linux Workstation."

# A collection of different objects
students = [ITStudent("Alice", 101, 1, "Admin"), CSStudent("Bob", 102, 1)]

for s in students:
    # Python dynamically binds the correct method at runtime!
    print(s.access_lab())
```

Summary

Class: The `class` keyword defines the blueprint.
Constructor: `__init__` sets up the initial state.
Self: The binding glue between the object and its attributes/methods.